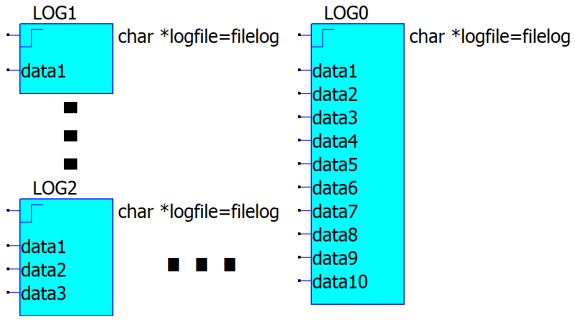
log
2.0

Features

- The user can store specific data into an external log file as the simulation is executing.
- Provided are 1- through 10-input versions of the log component.
- C-Block source included to allow for customizations.
- Allows logging of data across steps.
- You can use multiple instances of the component in the same simulation.
- This version is an improvement of my previous log V1.0 component. Major improvements:
    - All input versions use the same DLL (log.dll).
    - Selecting the number of inputs allows the symbol to grow or shrink. (This feature uses the LOOPUP<> feature in symbols)
    - Data saved can be triggered on Rising Edge, Fall Edge, Window High or Window Low.
    - Data can be saved as CSV or Binary formatted.



General description

The log component provides a simple interface from a Schematic/Netlist to log specific values into a named log file during the execution of the simulation.

This is similar to the post-simulation Waveform Analysis tool that uses .meas statements with the TRIG keyword. Using the Waveform Analysis tool, you can then direct it to export the requested data. With log component, you may be able to skip the Waveform Analysis phase or perform this phase at a later time with tools such as Excel with less extraneous data to parse.

When to use log component

If you have a simulation project that executes quickly, it probably has a relatively small .qraw file. The log component may be of little value. However, if you run long multi-dimensional stepped simulations, you may find log useful.

I developed a very data intensive multi-step transient simulation that required over 20 minutes to execute. All the time points are collected in the .qraw file. Once the simulation had completed, the .qraw file ended being over 28 GBytes big!!!

With this large .qraw file size, the .meas directives (there were 3 of them) took over 2 hours to complete into .meas results. Therefore, my total time to acquire the needed data on a multi-step set of parameters was about 2 ½ hours. This is because each .meas directive parses each time quanta collected in the .qraw to meet the criteria of the directive. With 3 directives, it parsed the .qraw file 3 times.

I created log to eliminate the post-simulation analysis phase and removed the .meas directives. Therefore, my resultant time to gain the needed data was only the time to execute the simulation (in this case, 20 minutes). With the CSV created, it only took about 10 minutes to parse in Excel for the required measurement results.

Even using the batch mode, with .qraw output turned off, I had another very extensive multi-step sim that took over 8 days to complete! Trying to store .qraw data would have exceeded my hard drive capacity in just an hour. I needed to run the QSpice sim in batch mode with the .qraw file creation disabled but using a log component to obtain only the needed data.

This allowed me to run multiple simulations with different sets of parameters much quicker.

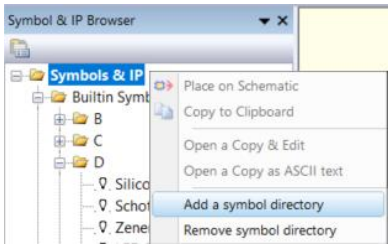
Special Note: On my computer (with multiple CPUs) I ran 4 instances simultaneously of QSpice each with a different parameter set. Each sim run took only slightly longer than if I had just run one.

log Setup

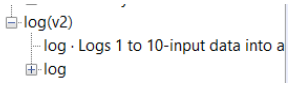
This log Component uses the QSpice symbol LOOKUP<> feature. This flexible feature allows the creation of the desired symbol with the required inputs using a sophisticated layering approach. The user only needs to drag the one top-level symbol from the “Symbols & IP Browser” and modify the String Attribute arguments to change the component’s operation and how it looks on the schematic.

To use this component, there are some setup requirements but this can be achieved fairly easily.

1. *Move this component's contents to your local project directory:*
   a. Copy or move the "log(v2)" directory of contents to where you normally locate your local simulation projects. This would normally be found in QSpice's default project directory "<username>\Documents\QSpice\"
2. *Set up QSpice to find the component:*
   a. Open QSpice and Right-click on the "Symbols & IP" level on the "Symbols & IP Browser". Select "Add a symbol directory".



   b. On the file explorer find and select where you stored the "log(v2)" directory and click on "Select Folder" button
   c. You'll find the "log(v2)" directory added to the "Symbol & IP" hierarchy



3. *Set up to use the LOOKUP<> feature: (This may require running as ADMIN for a short time)*
   a. Using Windows Explorer, find the installation directory of QSpice.
   b. If you've installed as ADMIN, then it should be at C:\Program Files\QSPICE.
   c. If you've installed as USER, then it should be at C:\Users\<username>\QSPICE.
   d. Select "New Folder" button and call it "Repository" in this installation directory. (You may be asked for ADMIN permission).
   e. In another Explorer window, **COPY** the "log" directory found in "log(v2)" to "Repository". This will place the lower-level symbols where QSpice can find them. (You may be asked for ADMIN permission).

There are efforts to improve the last step (3.). Reason: An official uninstallation of QSpice will delete the "Repository" directory and its' contents. Reinstalling QSpice would require you to repeat step 3.

Input-output connections

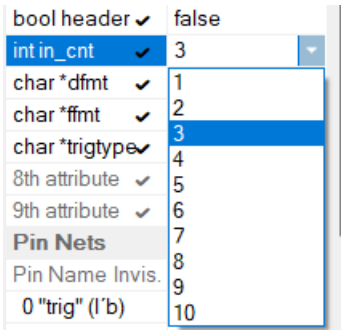
There are only hardware inputs but no hardware outputs on this component. This component does provide the logfile as the output.

datax

The number of inputs can be user-selected on the log Component. Up to 10 inputs can be selected. For example: log with 3 data inputs labelled data1, data2, and data3.



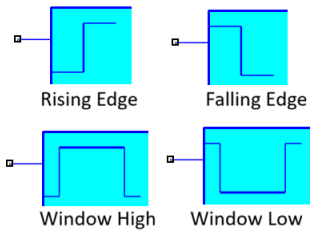
To change the number of inputs to use, open “Symbol Properties” (F3) and select the number from the “in_cnt” field. The symbol will then be populated with the desired inputs.



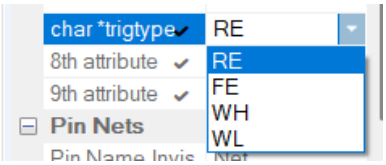
Each data input uses the default QSpice data type of “double” (64-bit floating). You can place signal into it since QSpice defines all node voltages as double. Note: The data type “double” is the default data type for node voltages.

trig

This input (binary) is the trigger for logging data. It logs the values on the data inputs into the logfile only on the selected criteria: Rising Edge (RE), Falling Edge (FE), Window High (WH) or Window Low (WL).



To change the trigger criteria, open “Symbol Properties” (F3) and select the criteria from the “trigtype” field. The symbol will then change to reflect the new selection.



Parameters and Settings

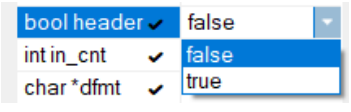
logfile

This parameter allows the assignment of the filename to use to log the data. Ex:
char *logfile=filelog // will store the logged data into “INSTANCENAME filelog<ffmt>”
Recommendations:

- You can use any legal file name. The file extension is added automatically to the end of the filename selected. The extension is based on the parameter “ffmt” selected.
- The logfile can be relative or absolute addressed. Leaving off any directory references assumes the file is located in the same directory as the simulation.
- If you use multiple instances of log in the same simulation, the INSTANCENAME (ie. LOG0) will be prepended to the logfile name to differentiate the data from each instance of the log component used.

header

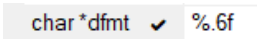
This parameter allows the user to assign a header to be placed at the top of the logfile. The default is *false* (OFF). This parameter is here to allow for clarity of the data in each column.



in_cnt

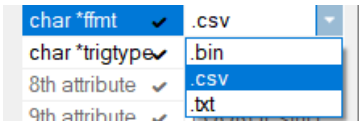
This parameter allows the user to assign the number of inputs to use. (See datax input explanation.) The symbol inputs available will change based on this selection.

dfmt



This parameter allows the user to determine the format of the data stored with the ability to determine precision. (Default: %.6f). This is a fprintf() format specifier. Any changes must adhere to legal format specifier rules. Here's a link to the format specifiers reference: https://cplusplus.com/reference/cstdio/fprintf/ Note: "float" specifiers are recommended ('f', 'e', 'g' or 'a'). Use of other specifiers may yield unpredictable results. For example, "%.6f" stores all data with 6-digits to the right of the decimal point (1µV of resolution). "%.3f" => 1mV resolution. "%.9f" => 1nV resolution. "%.12f" => 1pV resolution.

ffmt



This parameter allows the user to determine the file format of the data. (Default: .csv). File format **.csv** saves data in a text-based Comma Separated Value format commonly used by spreadsheet applications. The file format **.txt** is also an alias for **.csv** format.

Another file format is **.bin** This ffmt stores the data more compactly and may result in somewhat shorter simulation times. However, a special application is needed to convert **.bin** to **.csv** This tool can be provided upon request. (Courtesy of RDunn.)

trigtype

This parameter allows the user to configure the trigger criteria for capturing the data into the logfile. (See trig input explanation.)

LOOKUP<>

Leave the next two parameters alone. They are used to layer the symbol properties. "Don't look behind the curtain!" However, look if you're interested in creating your own symbols with the LOOKUP<> feature. That's where the magic happens!

Output Format of the logfile

The output format of the Binary (.bin) is simple. It can be determined by reading the log.cpp source code.

The output format of the CSV (.csv or .txt) very simple and straight-forward.

Here's an example of a few logged data points:

t	data1	data2	data3
0.100001	0.01	0.904837	0.999861
0.300001	0.03	0.740817	0.998751
0.500001	0.05	0.60653	0.996534

<= Here the header=1 was set.
Header (t=time of the log event; data'x'=data on input)

Implementation Limitations

This version of the log Component has the following limitations:

- This component logs the data on the inputs on the "trigtype" criteria selected. Timesteps recorded are determined by other circuit elements or QSpice directives.
- If you need to capture data in the same logfile across steps, no step value information is natively captured. Use extra input(s) with the step values assigned. These values will be logged in the file.
- If you would like to have separate logfiles for each step value, the step values can be encoded into the logfile name attribute parameter. See the Stepped Sim Example for the special QSpice "trick". This creates extra logfiles with the step values encoded into the file name.

Future Improvements

I have included the C-block source files (.cpp) and the symbol files (.qsym) so that it might be instructive or even improve this component. For example, you can create your own versions of log with more inputs.

Examples

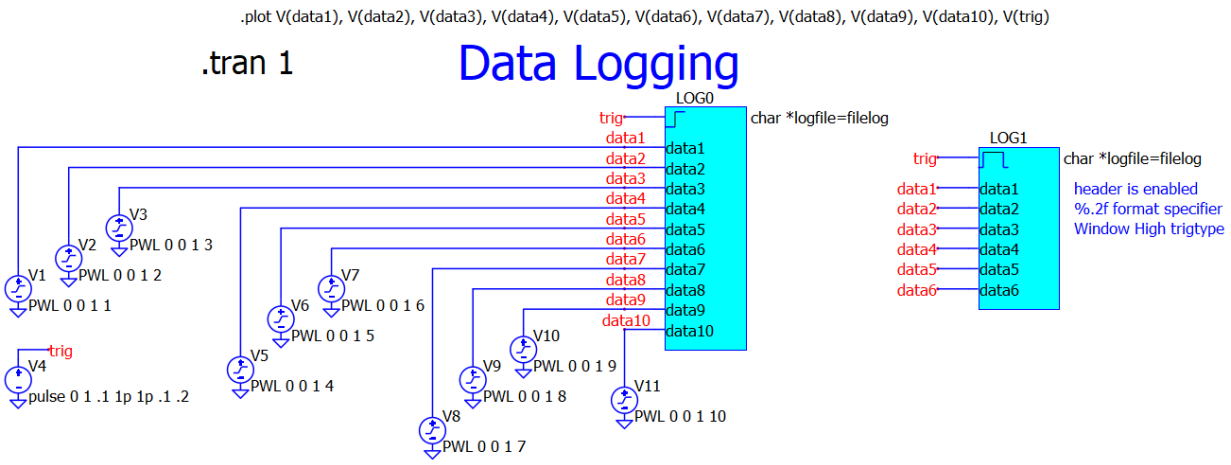
Here are very simple examples on how to use the log component. One is non-stepped and the other stepped.

Example goal

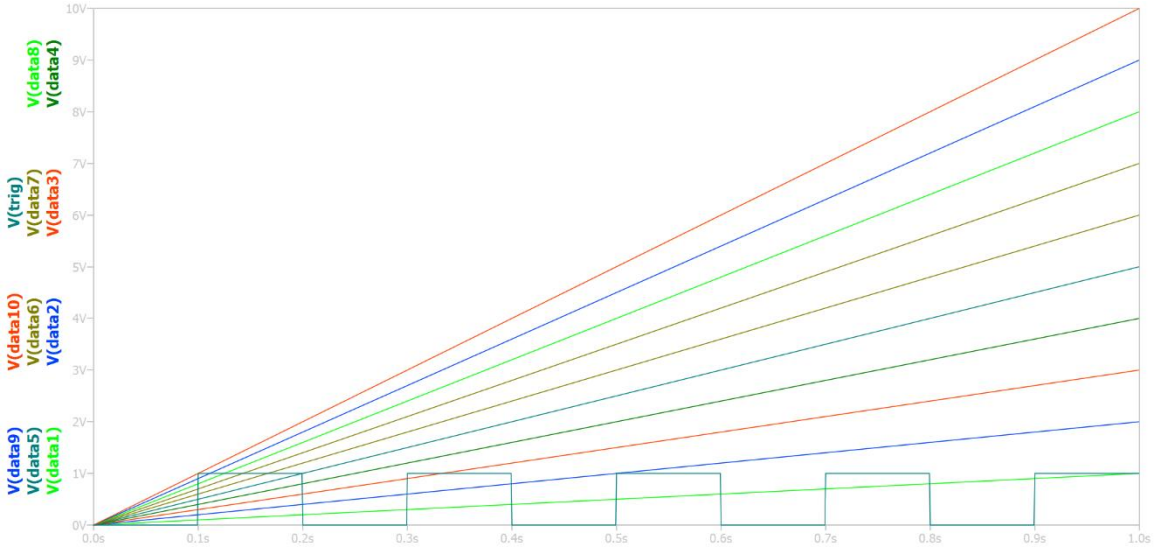
Generate multiple linear positive voltages and log data with a 50%DC 0.2s pulse.

Instructions: Simulation Specific – Non-stepped.

Here is an example log_test.qsch I've included:



When you run the simulation, you will get a plot of the selected signals.



You will get two logfiles: "LOG0 filelog.csv" and "LOG1 filelog.csv"

Open up the logfile "LOG0 filelog.csv" Note: header is disabled.

Table 1 - As seen in Excel

0.1	0.1	0.2	0.3	0.4	0.5	0.600001	0.700001	0.800001	0.900001	1.000001
0.3	0.3	0.6	0.9	1.2	1.5	1.800001	2.100001	2.400001	2.700001	3.000001
0.5	0.5	1	1.5	2	2.5	3.000001	3.500001	4.000001	4.500001	5.000001
0.7	0.7	1.4	2.1	2.8	3.5	4.200001	4.900001	5.600001	6.300001	7.000001
0.9	0.9	1.8	2.7	3.6	4.5	5.400001	6.300001	7.200001	8.100001	9.000001

Open up the logfile "LOG1 filelog.csv". Note:

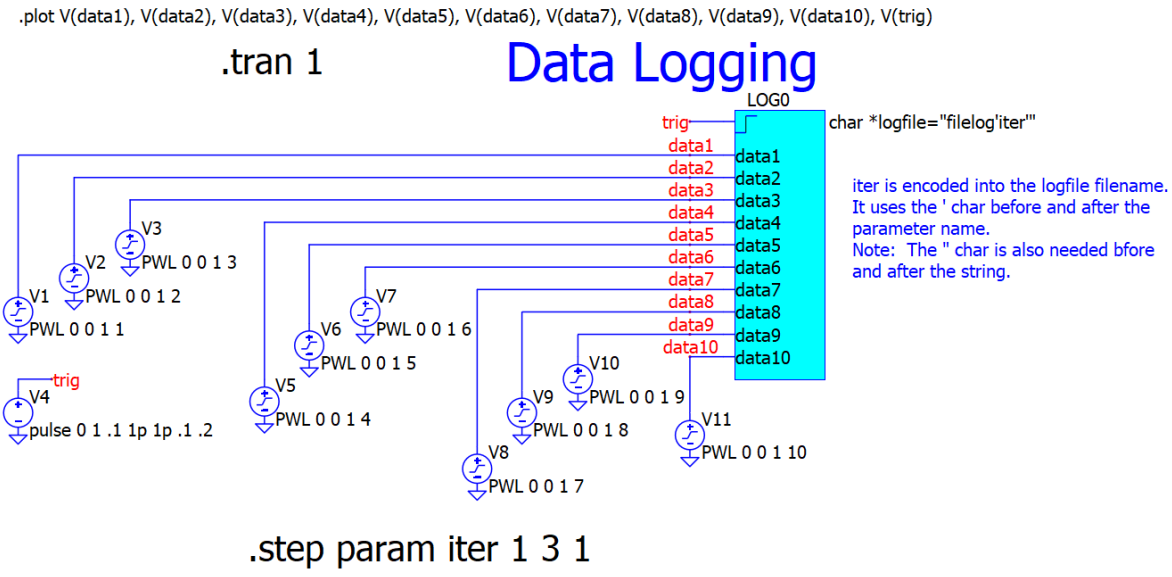
- header is enabled.
- trigtype=Window High => captures the data as long as the trig is High.
- dfmt=%.6f => to lower the data precision from the default.

Table 2 - As seen in Excel

t	data1	data2	data3	data4	data5	data6
0.1	0.1	0.2	0.3	0.4	0.5	0.600001
0.1	0.1	0.200001	0.300001	0.400001	0.500001	0.600002
0.100001	0.100001	0.200001	0.300002	0.400003	0.500003	0.600004
0.100001	0.100001	0.200003	0.300004	0.400006	0.500007	0.600009
0.100003	0.100003	0.200006	0.300009	0.400012	0.500015	0.600018
0.100006	0.100006	0.200012	0.300018	0.400025	0.500031	0.600037
0.100012	0.100012	0.200025	0.300037	0.40005	0.500062	0.600074
0.100025	0.100025	0.20005	0.300075	0.4001	0.500125	0.600149
0.10005	0.10005	0.2001	0.30015	0.4002	0.50025	0.600299
…	…	…	…	…	…	…

Instructions: Simulation Specific – Stepped.

- I modified the schematic of the non-step simulation to create a stepped version as shown below (log_test_step.qsch supplied):



Notice the additions and changes:

- The addition of the ".step param iter 1 3 1" directive.
- I added the value of 'iter' into logfile parameter for LOG0. This will generate a new log file with this in the file name for all steps with this iter value. Ie. "LOG0 filelog 1 .csv", "LOG0 filelog 2 .csv" and "LOG0 filelog 3 .csv",

Component Changes

Version	Description of changes	Reason for changes/impact
log 2.0	Improvements to “log” component. Recommend: Use this component instead.	Reuses the same log.dll. Uses LOOKUP<> feature to layer symbol properties. Adds many other selectable properties: • Supports up to 10 inputs • trigger type • file format • data format
log 1.0	Improvements to “logger” component.	Reuses the same log.dll.
logger 2.0	Eliminate the need of the “reset” input. Minor document improvements	Needs QSpice Jan 1, 2024 build
logger 1.0	first release of the component	

References

© CONSUL*TRON*, 2026 All Rights Reserved. CONSUL*TRON* allows PUBLIC use of the file.
CONSUL*TRON* allows public or commercial use of this product. Please use it for good.
DISCLAIMER: CONSUL*TRON* provides NO WARRANTY expressed or implied.
This product is intended for non-critical or non-safety use and can be used for educational purposes.